

The Ultimate Web Development Handbook



Loops in JavaScript

Loops allow you to execute a block of code multiple times, which is useful for tasks like iterating over arrays or repeating operations until a condition changes.

1. `for` Loop

Syntax

```
for (initialization; condition; finalExpression) {  
    // code to execute on each iteration  
}
```

Example

```
for (let i = 0; i < 5; i++) {  
    console.log(i); // prints 0, 1, 2, 3, 4  
}
```

- **initialization**: executed once before the loop starts (e.g., `let i = 0`)
 - **condition**: checked before each iteration; if `true` , the loop continues
 - **finalExpression**: executed at the end of each iteration (e.g., `i++`)
-

2. `while` Loop

Syntax

```
while (condition) {  
    // code to execute while condition is true  
}
```

Example

```
let count = 0;  
while (count < 3) {  
    console.log(count); // prints 0, 1, 2  
    count++;  
}
```

- The condition is evaluated before each iteration.
 - If the condition is initially `false`, the loop body may never run.
-

3. do...while Loop

Syntax

```
do {  
    // code to execute  
} while (condition);
```

Example

```
let num = 0;  
do {  
    console.log(num); // prints 0  
    num++;  
} while (num < 1);
```

- The loop body executes **at least once**, then the condition is checked.
-

4. `for...of` Loop

Purpose: Iterate over iterable objects (arrays, strings, etc.).

Syntax

```
for (variable of iterable) {  
    // code using variable  
}
```

Example

```
let colors = ["red", "green", "blue"];  
for (let color of colors) {  
    console.log(color);  
}
```

5. `for...in` Loop

Purpose: Iterate over the keys of an object.

Syntax

```
for (key in object) {  
    // code using key and object[key]  
}
```

Example

```
let person = { name: "Alice", age: 30 };  
for (let prop in person) {  
    console.log(prop + ": " + person[prop]);  
    // prints "name: Alice" and then "age: 30"  
}
```

Summary

- `for` : general-purpose loop with counter.
- `while` : loop based on a condition.
- `do...while` : condition checked after first execution.
- `for...of` : iterate over iterable values.
- `for...in` : iterate over object keys.

Control Flow in JavaScript

Control flow means **how your code runs step-by-step**, and how you can make decisions or repeat actions.

JavaScript runs code **from top to bottom**, but you can control the flow using:

- Conditional statements (`if` , `else` , `switch`)
 - Loops (`for` , `while` , `do...while`)
-

1. `if` , `else if` , and `else`

Use these to run code only if a condition is true.

```
let age = 18;

if (age >= 18) {
  console.log("You are an adult");
} else if (age >= 13) {
  console.log("You are a teenager");
} else {
  console.log("You are a child");
}
```

The first condition that is true will run, others will be skipped.

2. `switch` Statement

Use this when you have **multiple fixed cases** to check.

```
let day = "Monday";

switch (day) {
  case "Monday":
    console.log("Start of the week");
    break;
  case "Friday":
    console.log("End of the week");
    break;
  default:
    console.log("Midweek day");
}
```

- Use `break` to stop after each case.
 - `default` runs if no cases match.
-

3. Loops

Loops let you run code multiple times.

a. `for` Loop

Used when you know how many times to repeat.

```
for (let i = 1; i <= 5; i++) {
  console.log("Count:", i);
}
```

b. `while` Loop

Used when the number of repetitions is unknown.

```
let i = 1;
while (i <= 3) {
  console.log("While loop:", i);
}
```

```
i++;  
}
```

c. `do...while` Loop

Same as `while` , but runs at least once, even if the condition is false.

```
let i = 1;  
do {  
  console.log("Do while:", i);  
  i++;  
} while (i <= 3);
```

4. `break` and `continue`

- `break` = exit the loop immediately
- `continue` = skip the current iteration

Example:

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) continue; // skip 3  
  console.log(i);        // 1, 2, 4, 5  
}
```

Summary

- Use `if` , `else if` , and `else` to control logic.
- Use `switch` for multiple case checking.
- Use loops to repeat actions.
- Use `break` to stop loops, and `continue` to skip one turn.

break and continue in JavaScript

break Statement

The `break` statement is used to exit a loop prematurely, before the loop condition evaluates to false.

Syntax

```
break;
```

Example: Exiting a loop when a condition is met

```
for (let i = 0; i < 10; i++) {  
  if (i === 5) {  
    break; // exits the loop when i equals 5  
  }  
  console.log(i);  
}
```

Output:

```
0  
1  
2  
3  
4
```

Use Cases

- Exiting a `for`, `while`, or `do...while` loop early

`continue` Statement

The `continue` statement skips the current iteration of a loop and proceeds to the next one.

Syntax

```
continue;
```

Example: Skipping a loop iteration

```
for (let i = 0; i < 10; i++) {  
  if (i % 2 === 0) {  
    continue; // skips even numbers  
  }  
  console.log(i);  
}
```

Output:

```
1  
3  
5  
7  
9
```

Use Cases

- Skipping specific iterations based on a condition
 - Useful in filtering or avoiding certain values during iteration
-

Summary

Statement	Purpose	Effect
<code>break</code>	Exit the loop entirely	Stops loop execution
<code>continue</code>	Skip current iteration	Moves to the next iteration immediately

Both statements help in controlling loop execution flow more precisely based on conditions.